Fox and Hounds '45 - Evaluation Report

COSC 523 | 400 total simulations across four 2x2 conditions

Evaluation Summary

Each condition was run for 100 games. The random strategies selected uniformly from legal moves. The Fox AI used breadth-first search (BFS) to select a shortest available path, while the Hounds AI used depth-4 Minimax. A repeated board state occurring 100 times was classified as a draw.

Condition	Fox	Hounds	Draw	Avg. turns
Random Fox / Random Hounds	65	35	0	44.54
Random Fox / Minimax Hounds	0	100	0	7.75
BFS Fox / Random Hounds	75	25	0	15.48
BFS Fox / Minimax Hounds	0	77	23	56.54

Note: In the BFS Fox vs. Minimax Hounds condition, all 23 draws ended at 203 turns because the repeated-state termination rule was reached. The 77 completed Hound wins in that condition averaged 12.79 turns.

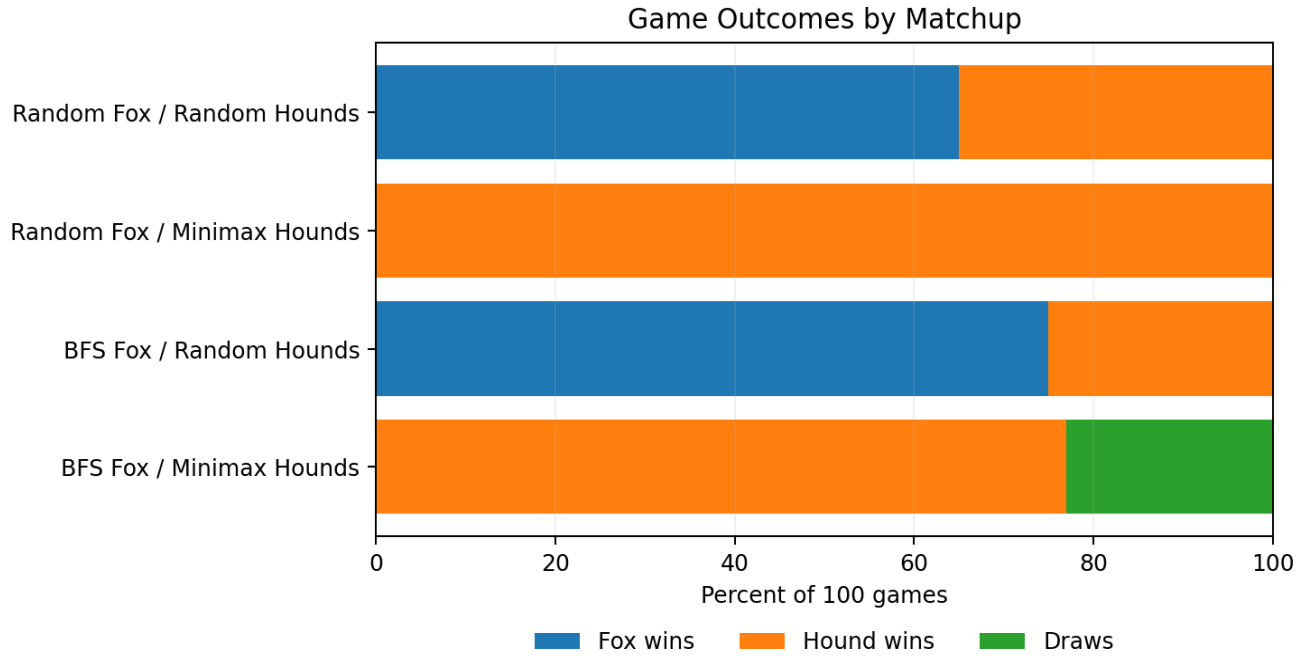


Figure 1. Outcome distribution for each 100-game condition. Because each condition contains exactly 100 games, counts are also percentages.

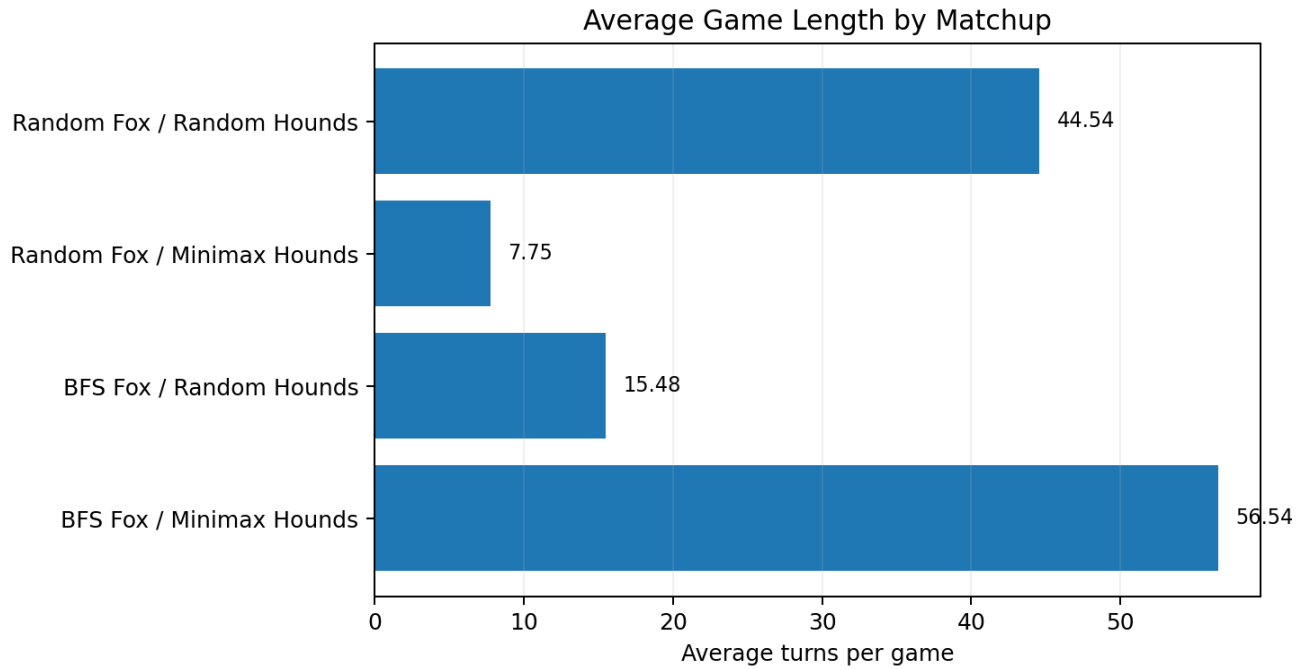


Figure 2. Mean game length for each condition. The BFS Fox vs. Minimax Hounds mean includes the 23 draws that terminated at 203 turns.

Overall Performance

The Minimax Hounds produced the strongest overall performance. Against the Random Fox, Minimax won all 100 games and did so quickly, with an average game length of 7.75 turns. The shortest-path Fox improved on the Random Fox when both faced Random Hounds: Fox wins increased from 65% to 75%, while average game length dropped from 44.54 to 15.48 turns. However, the shortest-path strategy was not enough to defeat Minimax. In the AI-vs-AI condition, the Hounds won 77 games, 23 games ended in draws, and the Fox did not win any games. This suggests that BFS was effective at finding efficient routes against a non-adversarial opponent, while Minimax was better able to anticipate the Fox's future moves and block progress. The AI-vs-AI average of 56.54 turns was driven by the draw condition; the 77 completed Hound wins averaged only 12.79 turns.

Design Decisions

I separated the game rules from the AI decision-making so that GameBoard manages legal moves, game state, winner detection, and rendering, while each AI strategy only chooses among valid actions. For the Fox, I used breadth-first search because each legal move has the same cost, so BFS can find a shortest path without the additional complexity of weighted-path algorithms. I did originally consider doing the Dijkstra algorithm for the fox AI strategy but realized it wouldn't have been an optimal AI strategy given that it utilizes weighted nodes. For the Hounds, I used depth-4 Minimax, treating the Hounds as the maximizing player and the Fox as the minimizing player. The evaluation function assigns large terminal scores for wins and losses, then scores non-terminal positions using the Fox's distance from the goal and the number of legal Fox moves. I also randomized ties between equally scored moves so the Hounds would not always choose the same equivalent action. Because some AI states can cycle indefinitely, I added a draw rule when the same complete board configuration occurs 100 times. This allowed every simulation to terminate while preserving repeated-state behavior as part of the evaluation rather than incorrectly counting those games as wins. I did not end up implementing the Alpha-Beta pruning as my solution technically worked for 100+ simulations, although it was painfully slow. I would like to eventually add this into my project to see the boost in performance. For the simulations, I ended up adding multi-threading capabilities so that I could run large volume game simulations in parallel to each-other. So, each game simulation is assigned a thread and will run across available CPUs to boost processing time. That's a whole lot better than running 400 simulations one after the other.

Please see the Demo.mp4 for a visualization of one of the game simulations.

I also pushed this project up to my Github profile just in case the uploaded files are missing anything!
https://github.com/trosnerMSU/fox_and_hounds/tree/main